

CSV Processor — Architecture

System Overview

[diagram — see online rendered version]

Kubernetes Pod

[diagram — see online rendered version]

CSV Upload — Sequence Diagram

[diagram — see online rendered version]

Infrastructure Choice

kops	Terraform / EKS
Control plane Self-managed on EC2	AWS-managed
Node groups On-demand + Spot + Mixed	On-demand + Spot
IRSA-hosted OIDC	Native EKS OIDC
See <code>/scripts/bootstrap.sh</code>	<code>terraform apply</code>

CSV Processor — Solution Design

Interview prep: see [INTERVIEW GUIDE.md](#) for a shorter, presentation-oriented walkthrough. Diagrams: [ARCHITECTURE.md](#).

Table of Contents

- 1. [Project Overview](#)
- 2. [Repository Structure](#)
- 3. [Architecture](#)
- 4. [Infrastructure Design](#)

5. [4.1 kops \(Primary\)](#)
 6. [4.2 Terraform / EKS \(Alternative\)](#)
 7. [4.3 Choosing Between kops and EKS](#)
 8. [Application Design](#)
 9. [Kubernetes Design](#)
 10. [Configuration Management — Ansible + Helm](#)
 11. [Autoscaling Strategy](#)
 12. [S3 Storage & Lifecycle](#)
 13. [Security Design](#)
 14. [Environment Matrix](#)
 15. [Quick Start](#)
-

1. Project Overview

The CSV Processor is a containerised Python/Flask web application that:

- Accepts CSV file uploads (SOH stock-on-hand export format)
- Parses each row and displays the results in the browser
- Uploads processed files to an S3 bucket
- Maintains a cluster-wide upload history via S3 (consistent across multiple replicas)

The platform runs on Kubernetes (kops self-managed or AWS EKS) with:

- An nginx sidecar serving static assets inside each pod
- Horizontal Pod Autoscaler and Cluster Autoscaler for elasticity
- IRSA (IAM Roles for Service Accounts) for credential-less S3 access from pods
- Ansible for environment-specific configuration management
- Helm for rendering and reusing Kubernetes manifests across environments

2. Repository Structure

Single monorepo — one `git clone https://github.com/asif-ahmedb/csv-processor` gives you everything.

```
csv-processor/                                ← monorepo root
├── .github/
├── workflows/
├── ci.yml                                     # CI – triggers only on csv-processor-app/** changes
├──
├── csv-processor-app/                         # Application source code and Docker image
├── web/
├── app.py                                     # Flask application
└── templates/index.html                       # Upload UI
```

```

■ ■ ■■■ static/css|js/           # Frontend assets
■ ■ ■■■ tests/test_app.py        # pytest suite
■ ■ ■■■ requirements.txt
■ ■■■ nginx/nginx.conf          # Standalone nginx config (non-K8s mode)
■ ■■■ docker/entrypoint.sh       # Standalone entrypoint
■ ■■■ Dockerfile
■ ■■■ sample-data/              # Example SOH CSV files
■
■■■■ csv-processor-infrastructure/ # AWS cluster provisioning – pick one path
■ ■■■ kops/                     # Self-managed Kubernetes on EC2
■ ■ ■■■ KOPS_QUICK_REFERENCE.md  # Full phase-by-phase kops command reference
■ ■ ■■■ settings.env.example     # Copy → settings.env, fill in account ID + region
■ ■ ■■■ scripts/                # bootstrap.sh · teardown.sh · configure-kubeconfig.sh
■ ■ ■■■ iam/                    # IAM policy templates (S3 + cluster-autoscaler)
■ ■ ■■■ manifests/              # Cluster spec, instance groups, autoscaler manifests
■ ■■■ terraform/                # Managed EKS alternative
■ ■■■ main.tf                   # VPC, EKS cluster, S3 bucket, IRSA role
■ ■■■ variables.tf
■ ■■■ outputs.tf                # s3_bucket_name + app_irsa_role_arn (consumed by Helm)
■ ■■■ terraform.tfvars.example
■
■■■■ csv-processor-k8s-assets/    # Kubernetes deployment assets (all environments)
■ ■■■ helm/csv-processor/        # Helm chart
■ ■■■ templates/                # Deployment, Service, HPA, Ingress, PVC, SA
■ ■■■ values.yaml                # Base defaults
■ ■■■ values-minikube.yaml       # Local dev overrides
■ ■■■ values-kops.yaml           # kops/AWS overrides
■ ■■■ values-eks.yaml            # EKS/Terraform overrides
■■■■ ansible/                   # Renders env-specific Helm values from inventory vars
■■■■ rendered/                  # Pre-rendered values-dev.yaml (Windows / no-Ansible fallback)
■■■■ minikube/                  # Local deploy scripts (deploy.sh · deploy.ps1)
■ ■■■ MINIKUBE_QUICK_REFERENCE.md # Full Minikube command reference
■■■■ DEPLOY_QUICK_REFERENCE.md   # Full kops + EKS Helm deploy command reference

```

3. Architecture

See **ARCHITECTURE.md** for diagrams: - System overview (CI/CD → K8s → S3) -
Kubernetes pod internals (initContainer + nginx + Flask) - CSV upload sequence diagram

4. Infrastructure Design

4.1 kops (Primary)

kops provisions a fully self-managed Kubernetes cluster on EC2. All resources (VPC subnets, security groups, IAM roles, ASGs, EC2 instances) are created and owned by kops.

Instance Groups:

Name	Role	Type	Min	Max	Notes
master-us-east-1a	Control plane	t3.small	1	1	Private subnet; runs etcd + API server
nodes-ondemand	Worker	t3.small	1	3	Guaranteed capacity; always-on baseline
nodes-spot	Worker	t3.small/medium	0	5	Pure spot; zero cost when idle
nodes-mixed	Worker	t3.small/medium	1	4	1 on-demand base + spot above; balanced cost/reliability

IRSA (IAM Roles for Service Accounts):

kops enables IRSA natively via `serviceAccountExternalPermissions` in the cluster spec and an S3-hosted OIDC discovery endpoint. The pod identity webhook injects `AWS_ROLE_ARN` and `AWS_WEB_IDENTITY_TOKEN_FILE` into pods for the configured service account. Helm also annotates the ServiceAccount with `eks.amazonaws.com/role-arn` (same pattern as EKS). Use `--set-string aws.accountId` when deploying — plain `--set` breaks 12-digit account IDs. No static credentials are stored in Kubernetes secrets.

Bootstrap sequence: 1. `cp settings.env.example settings.env` — fill in region + account ID 2. `./scripts/bootstrap.sh` — creates 3 S3 buckets (with lifecycle on processed/), IAM policy, renders cluster spec, runs `kops create` + `kops update`, configures kubeconfig for laptop API ELB access, waits for Ready nodes, installs cluster-autoscaler 3. `helm upgrade --install` from `csv-processor-k8s-assets`

Teardown:

```
./scripts/teardown.sh --yes # deletes cluster, IAM, all 3 S3 buckets
```

4.2 Terraform / EKS (Alternative)

Terraform provisions managed EKS as an alternative to kops. The AWS-managed control plane removes operational overhead (etcd backups, master upgrades) at the cost of less flexibility.

Resources created:

Resource	Module / Type
VPC + subnets + NAT	terraform-aws-modules/vpc/aws ~5.0
EKS cluster + node groups	terraform-aws-modules/eks/aws ~20.0
S3 bucket (encrypted, versioned)	aws_s3_bucket + lifecycle rules
IAM policy for S3 access	aws_iam_policy
IRSA role for app service account	terraform-aws-modules/iam/aws//iam-role-for-s ~5.0

Wire Terraform outputs into Helm:

```
helm upgrade --install csv-processor ./helm/csv-processor \
-f ./helm/csv-processor/values-eks.yaml \
--namespace csv-processor --create-namespace \
--set config.s3Bucket=$(terraform output -raw s3_bucket_name) \
--set aws.irsaroleArn=$(terraform output -raw app_irsarole_arn)
```

4.3 Choosing Between kops and EKS

Concern	kops	EKS
Control plane management	Self-managed (you own etcd, upgrades)	AWS-managed
Instance group flexibility	Full control — spot, mixed, custom AMIs	Managed node groups or Karpenter

IRSA mechanism	S3-hosted OIDC + pod identity webhook	Native EKS OIDC provider
Cost	EC2 only (no EKS control-plane fee)	+\$0.10/hr per cluster for control plane
Setup complexity	Higher (bootstrap script required)	Lower (Terraform handles it)
Recommended for	Custom K8s versions, full control	Production simplicity, AWS-managed SLAs

5. Application Design

The application is a single Python/Flask process (`app.py`) with no database dependency.

CSV Processing

Two input formats are supported automatically:

Format	Detection	Example
SOH headerless	First row: 3 fields, field 0 is all-digits	"211627629", "Purple Kaftan", "4900.0000"
Generic CSV with header	First row: any non-numeric	product_id, product_name, price

The parser (`_parse_csv`) returns a list of `{line, fields}` dicts. Blank lines are skipped. BOM (`\xef\xbb\xbf`) is stripped automatically.

S3 Integration

Operation	S3 Key Pattern	Purpose
-----------	----------------	---------

Upload processed CSV	<code>processed/{stamp}/{filename}</code>	Permanent archive
Write history entry	<code>history/{stamp}.json</code>	Cross-pod consistent history

`stamp` uses microsecond precision (`%Y%m%dT%H%M%S%fZ`) to prevent key collisions under concurrent load.

When `S3_BUCKET` is not set (local dev / minikube), S3 operations are silently skipped and history falls back to a local JSON file.

History Consistency

History is read from the S3 `history/` prefix when S3 is configured, giving all replicas a consistent view. When S3 is unavailable it falls back to the pod-local `processed.json`. This means:

- **With S3:** every pod shows the same history regardless of which pod handled which upload
- **Without S3** (minikube/dev): each pod shows only its own uploads (acceptable for local development)

Docker Image

The image contains both the Flask app and nginx. Two runtime modes:

Mode	Entry point	nginx starts?
Standalone (docker run)	<code>entrypoint.sh</code>	Yes — proxies on :8081
Kubernetes	<code>command:</code> <code>gunicorn</code> override in Deployment	No — nginx runs as a sidecar from the official image

The image runs as `www-data` (UID 33) in both modes.

6. Kubernetes Design

Helm Chart Structure

```

helm/csv-processor/
■■■ Chart.yaml
■■■ values.yaml           ← base defaults (all environments)
■■■ values-minikube.yaml  ← local dev (NodePort, standard SC, emptyDir)
■■■ values-kops.yaml      ← AWS kops (LoadBalancer, IRSA, emptyDir)
■■■ values-eks.yaml       ← AWS EKS (ClusterIP, IRSA, emptyDir)
■■■ templates/
  ■■■ deployment.yaml     ← 2-container pod + initContainer
  ■■■ service.yaml        ← ClusterIP or LoadBalancer
  ■■■ hpa.yaml            ← autoscaling/v2 HPA (CPU)
  ■■■ ingress.yaml        ← optional Ingress
  ■■■ pvc.yaml            ← optional PVC (disabled by default for AWS)
  ■■■ serviceaccount.yaml ← with optional IRSA annotation
  ■■■ configmap.yaml      ← env vars + nginx.conf

```

Key Design Decisions

Shared static files via emptyDir — The sync-static initContainer copies CSS/JS from the app image into a shared emptyDir volume. nginx reads from that volume for /static/ requests. This avoids NFS, a separate CDN, or duplicating static files in the nginx image.

nginx sidecar vs embedded nginx — nginx runs as a separate container within the same pod. Requests arrive at port 80 on the nginx container; non-static requests are proxied to 127.0.0.1:8080 (the app container). Static requests are served directly from the emptyDir volume without touching the app container.

HPA + no fixed spec.replicas — When autoscaling.enabled: true, spec.replicas is omitted from the Deployment so the HPA is the sole owner of replica count. Every helm upgrade would otherwise reset the HPA's scaling decisions.

PVC disabled for AWS — EBS (gp2/gp3) is ReadWriteOnce (one node). With autoscaling pushing pods to multiple nodes, a shared EBS PVC causes pods to hang in ContainerCreating. Since uploads are ephemeral staging (committed to S3 synchronously) and history is in S3, emptyDir is sufficient and correct.

Pod security context — runAsUser: 33 (www-data), allowPrivilegeEscalation: false, capabilities.drop: ALL. fsGroup: 33 ensures mounted volumes are writable by the app user.

7. Configuration Management — Ansible + Helm

Ansible manages the environment-specific variables that feed into Helm values. The separation is:

Layer

Owns

Ansible group_vars	App config (max_upload_bytes, s3_storage_class, aws_region, nginx tuning)
Ansible inventory (hosts.yml)	Environment identity (s3_bucket, image_repository, image_tag, hpa_min/max, IRSA ARN)
Helm values.yaml	Kubernetes defaults (resource limits, port numbers, probe config)
Helm overlay files	Cloud-provider specifics (storage class, service type, IRSA role name)

Ansible Role Flow

```

ansible-playbook -i inventory/prod playbooks/deploy.yml -e deploy_with_helm=true
  ■
  ■■■ role: csv_processor
    ■■ Render helm-values.yaml.j2 → rendered/values-production.yaml
    ■■ Render app-config.yaml.j2 → rendered/app-config-production.yaml
    ■■ (if deploy_with_helm=true) helm upgrade --install
    -f rendered/values-production.yaml

```

The default (`deploy_with_helm: false`) renders only — useful for review before applying or for GitOps workflows where the rendered file is committed and applied by a separate process.

8. Autoscaling Strategy

Cluster Autoscaler

The Cluster Autoscaler runs as a Deployment in `kube-system` and uses ASG tag-based auto-discovery. All three worker instance groups are tagged:

```

k8s.io/cluster-autoscaler/enabled = "true"
k8s.io/cluster-autoscaler/csv-processor.k8s.local = "owned"

```

When pods are unschedulable (pending), the CA scales up the cheapest available node group. When nodes are underutilised, it scales down (respecting PodDisruptionBudgets).

Scale-up priority: mixed group (on-demand base guaranteed) → spot group (cheapest) → on-demand group (fallback).

Horizontal Pod Autoscaler

HPA autoscaling/v2 scales the `csv-processor` Deployment between `minReplicas` and `maxReplicas` based on average CPU utilisation:

Environment	Min	Max	Target CPU
minikube	1	4	70%
kops (prod)	2	10	70%
EKS (prod)	2	20	65%

CPU-based scaling is triggered by CSV parsing workload. Each gunicorn worker (2 per pod) is CPU-bound during parsing.

9. S3 Storage & Lifecycle

Bucket Structure

```
csv-processor-processed-{account_id}/
  processed/
    {stamp}/
      {filename}.csv      ← uploaded CSV file
    history/
      {stamp}.json        ← entry metadata (row_count, filename, s3_uri, ...)
```

Lifecycle Policy (kops and Terraform)

Applied to the `processed/` prefix only (both paths — `kops bootstrap.sh` calls `aws s3api put-bucket-lifecycle-configuration`; Terraform uses `aws_s3_bucket_lifecycle_configuration`):

Transition	Days	Storage class
Current version → Glacier	30	GLACIER
Current version → Deep Archive	180	DEEP_ARCHIVE

Current version → Expire	365	—
Noncurrent versions → Glacier	90	GLACIER
Noncurrent versions → Expire	365	—

History entries (`history/` prefix) are not subject to lifecycle transitions — they are small JSON documents (< 1 KB each) that should remain accessible for the application.

S3 Security

Control	Setting
Public access	Fully blocked (<code>BlockPublicAcls</code> , <code>RestrictPublicBuckets</code>)
Encryption	AES-256 server-side encryption (SSE-S3)
Object ownership	<code>BucketOwnerEnforced</code> (ACLs disabled)
Versioning	Enabled
Access control	IRSA only — no static credentials

10. Security Design

Credential Management — IRSA

No AWS credentials are stored in Kubernetes Secrets or environment variables. Instead:

1. The cluster exposes an OIDC endpoint (S3-hosted for kops, native for EKS)
2. The app's ServiceAccount is annotated with the IAM role ARN
3. The pod identity webhook injects a short-lived projected service account token and the role ARN as environment variables
4. boto3 automatically exchanges the token for temporary STS credentials

The IAM policy grants least-privilege access: `PutObject`, `GetObject`, `DeleteObject`, `AbortMultipartUpload` on objects, and `ListBucket` + `GetBucketLocation` on the bucket. On kops, `ListBucket` is further scoped to the `processed/*` and `history/*` prefixes.

Container Security

Control	Value
Run as user	UID 33 (www-data)
Run as non-root	true
Allow privilege escalation	false
Linux capabilities	All dropped
Pod fsGroup	33 (mounted volumes writable by www-data)

Network Security

- Masters and worker nodes run in **private subnets** — not directly reachable from the internet
- Only the API load balancer (kops) and the app service load balancer are public-facing
- The kops cluster spec sets `kubernetesApiAccess` and `sshAccess` — restrict these CIDRs to your office/VPN in production

11. Environment Matrix

Feature	minikube	kops (AWS)	EKS (AWS)
Values file	<code>values-minikube.yaml</code>	<code>values-kops.yaml</code>	<code>values-eks.yaml</code>
Service type	<code>NodePort : 30080</code>	<code>LoadBalancer</code>	<code>ClusterIP</code>
Storage class	<code>standard</code>	<code>gp2 (disabled)</code>	<code>gp3 (disabled)</code>
Persistence	<code>enabled (for dev)</code>	<code>disabled (emptyDir)</code>	<code>disabled (emptyDir)</code>
S3	Optional (MinIO/localstack via <code>S3_ENDPOINT_URL</code>)	Required	Required

IRSA	Not needed	kops pod identity webhook	EKS OIDC
Autoscaling	min 1 / max 4	min 2 / max 10	min 2 / max 20
Node groups	single	ondemand + spot + mixed	ondemand + spot
Image registry	barbhua786/csv-processor	barbhua786/csv-processor	ECR (set in prod inventory)

12. Quick Start

Option A — kops on AWS

```
# 1. Prerequisites: aws cli, kops, kubectl, helm, envsubst
#
# 2. Configure
cd csv-processor-infrastructure/kops
cp settings.env.example settings.env
# Edit settings.env: set AWS_REGION and AWS_ACCOUNT_ID
#
# 3. Bootstrap cluster (~15 min)
chmod +x scripts/*.sh
./scripts/bootstrap.sh
#
# 4. Deploy application
cd ../../csv-processor-k8s-assets
source ../csv-processor-infrastructure/kops/settings.env
helm upgrade --install csv-processor ./helm/csv-processor \
  -f ./helm/csv-processor/values-kops.yaml \
  --namespace csv-processor --create-namespace \
  --set config.s3Bucket="${PROCESSED_BUCKET}" \
  --set-string aws.accountId="${AWS_ACCOUNT_ID}"
```

```
# 5. Get app URL
kubectl get svc -n csv-processor csv-processor
```

Option B — Terraform / EKS on AWS

```
# 1. Prerequisites: terraform, aws cli, kubectl, helm

# 2. Provision infrastructure (~10 min)
cd csv-processor-infrastructure/terraform
cp terraform.tfvars.example terraform.tfvars
terraform init && terraform apply

# 3. Configure kubectl
aws eks update-kubeconfig --name $(terraform output -raw eks_cluster_name) --region us-east-1

# 4. Deploy application
cd ../../csv-processor-k8s-assets
helm upgrade --install csv-processor ./helm/csv-processor \
  -f ./helm/csv-processor/values-eks.yaml \
  --namespace csv-processor --create-namespace \
  --set config.s3Bucket=$(terraform -chdir=../csv-processor-infrastructure/terraform output -raw s3_bucket) \
  --set aws.irsRoleArn=$(terraform -chdir=../csv-processor-infrastructure/terraform output -raw app_irs_role_arn)

# 5. Get app URL (via Ingress or port-forward)
kubectl port-forward -n csv-processor svc/csv-processor 8080:80
```

Option C — Minikube (Local)

See [MINIKUBE QUICK REFERENCE.md](#) for the full command reference.

```
minikube start
kubectl config use-context minikube
cd csv-processor-k8s-assets/minikube
./deploy.sh
kubectl port-forward -n csv-processor svc/csv-processor 8080:80
# http://localhost:8080
```

Option D — Ansible (renders values + optionally deploys)

```
cd csv-processor-k8s-assets/ansible
# Install collections
ansible-galaxy collection install -r requirements.yml

# Render values only (review before applying)
```

```
ansible-playbook -i inventory/dev playbooks/deploy.yml  
  
# Render + deploy in one step  
ansible-playbook -i inventory/prod playbooks/deploy.yml -e deploy_with_helm=true
```

Teardown (kops)

```
cd csv-processor-infrastructure/kops  
./scripts/teardown.sh --yes
```

Teardown (Terraform)

```
cd csv-processor-infrastructure/terraform  
terraform destroy
```